A3 - React Frontend

- Due Sunday by 11:59p.m.
- Points 200

For the term project, you are to implement a React frontend for the temporary staffing platform and connect it to the backend server that you built during Assignment 2. The deliverable is a fully functional website that is deployable, presentable, self-explanatory, smooth, and visually clear. The system should support the complete workflow for regular users, businesses, and administrators, including role-based access control, state-dependent actions, and time-sensitive interactions such as qualification review, job matching, and negotiation.

You can find general information about the project and its features from the **Assignment 2 handout (https://q.utoronto.ca/courses/416272/assignments/1655557)** , and you should continue to refer to it for API definitions and business rules.

INTERVIEW BOOKING PAGE IS UP:
https://outlook.office.com/book/CSC309Winter2026@bookings.office365.utoronto.ca/?ismsaljsauthenabled (https://outlook.office.com/book/CSC309Winter2026@bookings.office365.utoronto.ca/?ismsaljsauthenabled)

Setup

1. Log onto MarkUs and go to **CSC309**. You have been assigned the group you were in for A2. Notify the course staff if group adjustments are needed, e.g., you need a new team or teammate.
2. ONLY ONE PERSON IN THE GROUP SHOULD DO THIS:
   - Check out the **A3 repository** from MarkUs. There should be 2 empty files (WEBSITE and INSTALL), two directories named **frontend** and **backend** and a Makefile. You may add any number of files to both directories, but do not add anything to the root directory.
   - Copy everything from your **A2** directory into the `backend` directory in **A3**.
   - Make sure the backend code is working. Then, commit everything.
3. OTHER GROUP MEMBERS SHOULD DO THIS AFTER STEP 2:
   - Check out the **A3 repository** from MarkUs.

Requirements

You are to implement the frontend of the temporary staffing platform using React.js. For this milestone, your website must be deployed and publicly available. To simplify the scope, you are not required to account for concurrency or high-volume scalability; it is acceptable to use SQLite3 for your backend and assume that only a single user will be using your website at one time. Furthermore, you do not need to support data persistence across redeployments. Please be aware that most PaaS providers utilize

ephemeral file systems, meaning any local files, including your SQLite database and any user uploads, will be wiped whenever the backend restarts or is updated.

Documentation

1. **INSTALL**: This document should include a detailed description of any preparation needed for your code to run in a new environment. Include:
   - Prerequisite packages to install.
   - How to set up the backend and frontend.
   - Information on how to deploy your website.
     - The backend server can run on the same (virtual) machine as the web server for the frontend, and is preferred for simplicity.
     - **Note:** You do not have to share credentials for the hosting space, but you should outline the environment (e.g., a machine running Ubuntu 24.04).
2. **WEBSITE**:
   - This text file should ONLY contain the **URL** to your publicly hosted website. It should not contain anything else.

Frontend

Place your React project in the `frontend` directory by running the following commands in the `frontend` directory:

```
# You will be asked: "Current directory is not empty. Please choose how to proceed:" , to which you should select "Ignore files and continue".
npm create vite@latest . -- --template react
npm install
```

You may use React 18 or above, and *you may give this project any name you wish* (please avoid offensive names).

User Interface

Your frontend must present a complete interface for the temporary staffing platform. The system includes public views, authentication workflows, and role-dependent features for regular users, businesses, and administrators. The interface should be designed so that users can understand system state clearly, especially when actions are time-sensitive or restricted by role.

Navigation should be intuitive, responsive, and consistent across the site. Users should be able to move naturally between major workflows without manually editing URLs.

A navigation bar or equivalent persistent interface element must be present. It should provide access to major sections of the system, account-related actions, and any role-specific workflows that are currently available. Different roles should only see features they are allowed to use.

This system is fundamentally state-driven. A good interface should help users understand:

- what they can do now
- what they cannot do now
- why an action is unavailable

A clean interface that makes system constraints obvious will score better than one that merely exposes raw API functionality.

Basic Requirements

- Users should never need to manually edit URLs.
  - Use React Router so browser navigation works naturally.
- System state should always be visually obvious, especially:
  - whether a job is open / filled / expired / canceled / completed
  - whether negotiation is active
  - whether qualification is pending / approved / rejected
- Time-sensitive actions should visibly show deadlines or countdown timers.
- **Pagination must be implemented for every list endpoint.**
- Long lists should support sorting and filtering.

Detailed Requirements

The items listed below do not necessarily correspond one-to-one with standalone routes. A feature may be implemented as a page, component, modal, pop-up, or persistent interface element, provided the workflow remains clear.

Your frontend should include:

<u>Authentication</u>

A view or set of views that supports:

- account creation for regular users
- account creation for business users
- login using email and password
- account activation, if required by your backend
- password reset request
- password reset completion using a valid token

<u>Public Pages</u>

- A public landing page that introduces the platform and helps users navigate to the appropriate workflow, e.g., account registration and login.
- A public view for displaying the list of businesses.
- A public view for displaying a specific business profile, including:
  - business information
  - location

  - avatar
  - biography

Publicly accessible views should expose only information that is available to unauthenticated users.

Regular Users

- A view that displays the authenticated user's profile, including:
  - personal information
  - approved qualifications
  - uploaded avatar and resume
  - biography
  - availability status
- A view that allows the user to edit profile information.
- A view that allows the user to upload or replace:
  - avatar
  - resume
  - qualification documents
- A view that displays all visible position types and allows the user to:
  - create qualification requests
  - view qualification status
  - revise and resubmit qualifications
- A view that displays all qualification requests created by the user, including:
  - current status
  - last updated time
  - associated position type
- A view that displays available job postings for which the user qualifies, including:
  - business name
  - position type
  - salary range
  - work time window
  - optional travel distance and ETA
  - sorting, filtering, and pagination
- A view that displays details of a single job posting and allows:
  - expressing interest
  - withdrawing interest
- A view that displays current invitations received from businesses.
- A view that displays jobs the user is currently interested in, clearly indicating:
  - whether mutual interest has been reached
  - whether negotiation can be initiated
- A view that displays current confirmed work commitments and past filled, completed, or cancelled jobs.

Business Users

- A view that displays the authenticated business profile
  - Can be nearly identical to the public business view
- A view that allows editing business profile information.
- A view that allows uploading or replacing business avatar.
- A view that allows verified businesses to create new job postings.
- A view that displays all jobs created by the business, including:
  - status
  - worker if filled
  - filters, sorting, and pagination
- A view that displays a single job posting and allows:
  - editing if still allowed
  - deletion if allowed
- A view that displays discoverable candidates for a specific job, including:
  - qualification summary
  - whether already invited
  - Alternatively, already invited candidates may simply be excluded from this list.
- A view that displays detailed candidate information for a job, including:
  - qualification note
  - qualification document
  - resume
  - biography
- A view that allows expressing interest in candidates.
  - This may be implemented as a separate view, a pop-up, or integrated with the search or candidate detail view.
- A view that displays users who have expressed interest in a specific job posting, including whether mutual interest has been reached.
- A view that allows initiating negotiation where permitted.
- A view that allows marking a worker as no-show during the valid work interval.

Administrators

- A view that displays all regular users with:
  - filters
  - pagination
  - suspension controls (suspend or unsuspend)
- A view that displays all businesses with:
  - filters
  - verification controls (verify or unverify)
- A view that displays all position types and allows:
  - creation

  - editing
  - hiding
  - deletion where permitted
- A view that displays qualification requests requiring admin attention.
- A view that allows reviewing a qualification request and changing status:
  - approved
  - rejected
- A view that allows changing system-wide configuration values:
  - reset cooldown
  - negotiation window
  - job start window
  - availability timeout

Negotiation UI

Because negotiation is exclusive and time-sensitive, your frontend should clearly surface active negotiation state whenever relevant.

The interface should support:

- countdown timer
- negotiation messages
- accept / reject controls

It should also clearly communicate:

- who the negotiation is with
- whether each side has accepted
- whether the negotiation has expired

A user involved in negotiation should not be allowed to accidentally navigate into conflicting actions without clear feedback.

Backend

You are free to make any changes to your backend server to implement any additional features you want. At bare minimum, you will need to add the `cors` package to your backend so that your React development server can access the backend.

```
const cors = require('cors');
const app = express();

// Set up cors to allow requests from your React frontend
app.use(cors({
origin: 'http://localhost:5173',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
```

```
  credentials: true
});
```

Pre-populated Database

To help the TA and industry partner test and grade your website and improve your presentation, you should write a script to seed your database (remember to check in this script).

Your seeded database must include:

- At least 20 regular users with the following email address:
  - `regularX@csc309.utoronto.ca` where X is a number from 1 to your maximum number of regular users.
- At least 10 businesses with the following email address:
  - `businessX@csc309.utoronto.ca` where X is a number from 1 to your maximum number of businesses.
- At least 1 administrator
  - `adminX@csc309.utoronto.ca` where X is a number from 1 to your maximum number of administrators.
- At least 10 position types
- At least 30 sample job postings across different states where appropriate
- At least 20 qualification requests with a mix of statuses
- Enough interests, invitations, negotiations, and filled jobs to demonstrate the major workflows of the system
- Enough data on every page that displays a list of objects to trigger pagination

Also, note the following:

- All account passwords should be set to `123123`. You may have to temporarily break your password requirement to allow for this.
- The data does not have to be original or real, but it should look realistic.
- Ensure your database and media files are not too large, so the website still loads quickly.
- We will test pagination for every page that displays a list of objects, so make sure there is enough data to produce at least 2 pages where pagination is required.
- Your seeded data should make it easy for a marker to test different roles and system states without having to create accounts manually.

Grading

Please use the **CSC309 Booking Page (https://outlook.office.com/book/CSC309Winter2026@bookings.office365.utoronto.ca/?ismsaljsauthenabled)** to reserve a 1-hour grading session with a TA. Booking this session is mandatory. Failure to do so will result in a zero for the entire team, regardless of what is submitted on MarkUs. All team members must attend the grading session. Absent members will not receive any marks.

By default, the grading session will be in person, in room **BA 2270**. However, please email your TA to confirm the location or set up an alternative location. In-person availability is strongly encouraged, i.e., we discourage grading virtually unless it's absolutely necessary.

Each group should book only one time slot. Booking multiple slots may result in disciplinary action of -20% A3 mark for the entire group. To avoid accidental double booking, designate one team member to handle scheduling.

Grading interviews must be completed before the final exam on April 15th (9:00 AM – 12:00 PM).

Individual Contribution

Each team member is expected to contribute fairly to the project. We will assess participation by reviewing commit histories to evaluate individual contributions, and asking detailed technical questions about the code during the interview. While the group will receive a single mark, individual marks may be adjusted if we find significant disparities in contributions.

Preparation

To ensure your work is properly evaluated, practice demonstrating your website before the session. Your interview will conclude exactly at the one-hour mark, and any features not presented within this time frame will not be graded.

Rubrics

The total mark for this assignment is 200. We will post the detailed grading guideline later.

Mark Breakdown

The grading for each page is broken down as follows:

Functionality (80%)

The page should correctly support the intended workflow. This includes the normal expected use of the feature, as well as reasonable error/exception handling such as invalid input, empty results, loading states, actions that are unavailable, and access attempts that should be denied based on system state or user role. Your frontend should clearly handle cases where a user is not permitted to perform an action or access a page.

Aesthetics (20%, with up to 10% bonus)

Each page is expected to meet baseline visual standards, including readable layout, consistent spacing, proper alignment, and clear presentation of information.

Additional visual sophistication may help strengthen your overall score, for example through refined layout, stronger visual hierarchy, polished responsiveness, or thoughtful interaction design. However, the

maximum mark for any page **remains at 100%**, so functionality should always be your first priority. The bonus mark only serves as limited flexibility to compensate for minor functionality issues, e.g., your page shows a progress bar for uploading files, but you did not handle maximum file size limit correctly.

As general guidance:

- A page that is visually clear and well presented can still receive partial marks even if functionality is incomplete.
  - This is especially important for list pages (e.g., job listings, qualification listings, etc), which include sorting, filtering, and pagination.
- A page that correctly supports the expected workflow can receive most of the marks even if the design is simple.
- Full marks require both correctness and a clear, well-presented user experience.

Submission

Submit your work to your group's git repository on MarkUs to the **A3** repository. Note that the autotester will only check the WEBSITE text file to see if the URL is accessible. This assignment will be graded manually through an interview session with a TA. Please verify the availability of your teammates, then reserve a time slot through the **CSC309 Booking Page (https://outlook.office.com/book/CSC309Winter2026@bookings.office365.utoronto.ca/?ismsaljsauthenabled)**. All team members must show up to the grading session to receive marks.

Competition

To recognise outstanding implementations, we will hold a small showcase for the best websites built for this project.

Our industry partner, the **Chinese Dental Hygienist Association in Ontario (https://www.cdhao.ca/)**, will review selected submissions and choose their favourite implementation. The organization works with dental professionals across Ontario and is interested in exploring technology that improves communication and staffing coordination in the dental field.

The winning team will have the following options regarding the future of their code:

- **Handover**: the team may choose to provide their code to the organization for further refinement, but decline participation in future development.
- **Continuation**: the team may choose to continue developing the project, potentially as a follow-up project (for example through CSC494), subject to eligibility and mutual interest.

Participation in this showcase is optional. A team may opt out by informing the course instructor via email. Teams that opt out will still be graded normally, but their project will not be considered for the showcase.

Opting in or out of the showcase **will not affect your grade** for the assignment.

Academic Integrity

You are permitted to use open-source packages, images and code snippets from the Internet, including AI-generated content (e.g., ChatGPT), as long as you provide proper attribution.

- **Packages**: List any additional packages you use in the `INSTALL` document and include instructions on how to install and use them.
- **Code Snippets**: If you incorporate external code, cite the source directly as a comment above the relevant code.
- **Assets**: Clearly attribute any external images, fonts, or other assets at the point of import in your source code.
- **Important:** Sharing code with other teams is strictly prohibited. Submitting work that contains identical or highly similar passages of code without proper attribution will be considered an academic offense. Ensure that all external contributions are properly credited to maintain academic integrity.